

Ядром любой базы данных является модель данных. **Модель данных** – это совокупность структур данных и операций их обработки. С помощью модели данных могут быть представлены информационные объекты и взаимосвязи между ними. Рассмотрим три основных типа моделей данных: иерархическую, сетевую и реляционную.

Иерархическая модель данных представляет собой совокупность элементов данных, расположенных в порядке их подчинения и образующих по структуре перевернутое дерево (рис. 1). К основным понятиям иерархической модели данных относятся: уровень, узел и связь. Узел – это совокупность атрибутов данных, описывающих информационный объект.

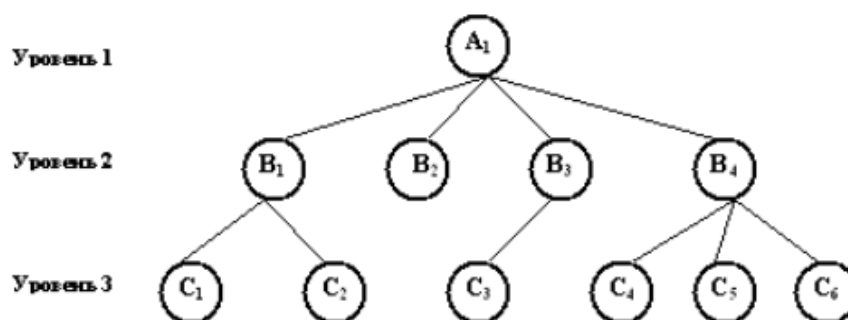


Рис. 1. Графическое изображение иерархической структуры данных

Иерархическая структура должна удовлетворять следующим требованиям:

- каждый узел на более низком уровне связан только с одним узлом, находящимся на более высоком уровне;
- существует только один корневой узел на самом верхнем уровне, не подчиненный никакому другому узлу;
- к каждому узлу существует ровно один путь от корневого узла.

Иллюстрация использования иерархической модели для построения базы данных «Институт» представлена на рис. 2.

Преимущества

Отношения родитель-потомок обеспечивают целостность данных. Каждое изменение, сделанное в родительской таблице, автоматически обновляется и в дочерней таблице.

Еще одно преимущество иерархических баз данных — высокий уровень безопасности.

Недостатки

Жесткая иерархия усложняет работу с данными. Если родительская и дочерняя таблицы не связаны между собой, то добавление новой записи в дочернюю таблицу усложняется, потому что ее необходимо добавить и в родительскую таблицу.

Изменение структуры БД приводит к изменению всех прикладных программ, которые с ней работают.

Иерархическая БД поддерживает отношения один к многим и не поддерживает отношения многие к многим.



Рис. 2. Пример иерархической структуры данных

Сетевая модель данных основана на тех же основных понятиях (уровень, узел, связь), что и иерархическая модель, но в сетевой модели каждый узел может быть связан с любым другим узлом. На рис. 3 схематически изображена сетевая структура организации данных.

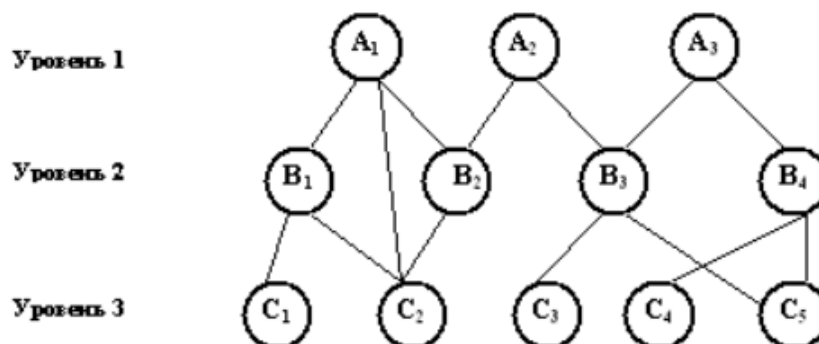


Рис. 3. Графическое изображение сетевой структуры данных

Примером сетевой структуры может служить структура базы данных, содержащей сведения о студентах, занимающихся в спортивных секциях. Возможно участие одного студента в нескольких секциях, возможно также участие нескольких студентов в одной секции. Графическое представление описанной в примере сетевой структуры, состоящей из двух типов информационных объектов, показано на рис. 4.



Рис. 4. Пример сетевой структуры данных

Сетевая модель данных имеет более гибкую структуру, чем иерархическая и поддерживает отношения «многие ко многим». Но быстро становится слишком сложной и неудобной для управления.

Реляционная модель данных использует организацию данных в виде двумерных таблиц. Каждая такая таблица, называемая **реляционной таблицей** или **отношением**, представляет собой двумерный массив и обладает следующими свойствами:

- все столбцы в таблице однородные, т.е. все элементы в одном столбце имеют одинаковый тип и максимально допустимый размер;
- каждый столбец имеет уникальное имя;
- одинаковые строки в таблице отсутствуют;
- порядок следования строк и столбцов в таблице не имеет значения.

Основными структурными элементами реляционной таблицы являются поле и запись (рис. 5). **Поле** (столбец реляционной таблицы) – элементарная единица логической организации данных, которая соответствует конкретному атрибуту информационного объекта. **Запись** (строка реляционной таблицы) – совокупность логически связанных полей, соответствующая конкретному экземпляру информационного объекта.

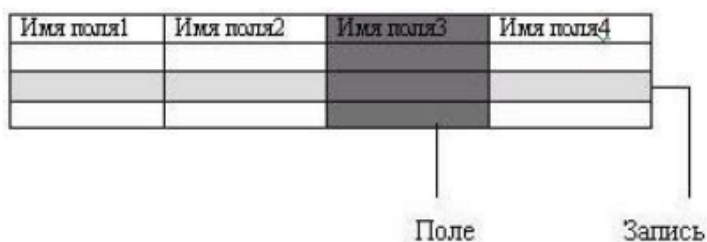
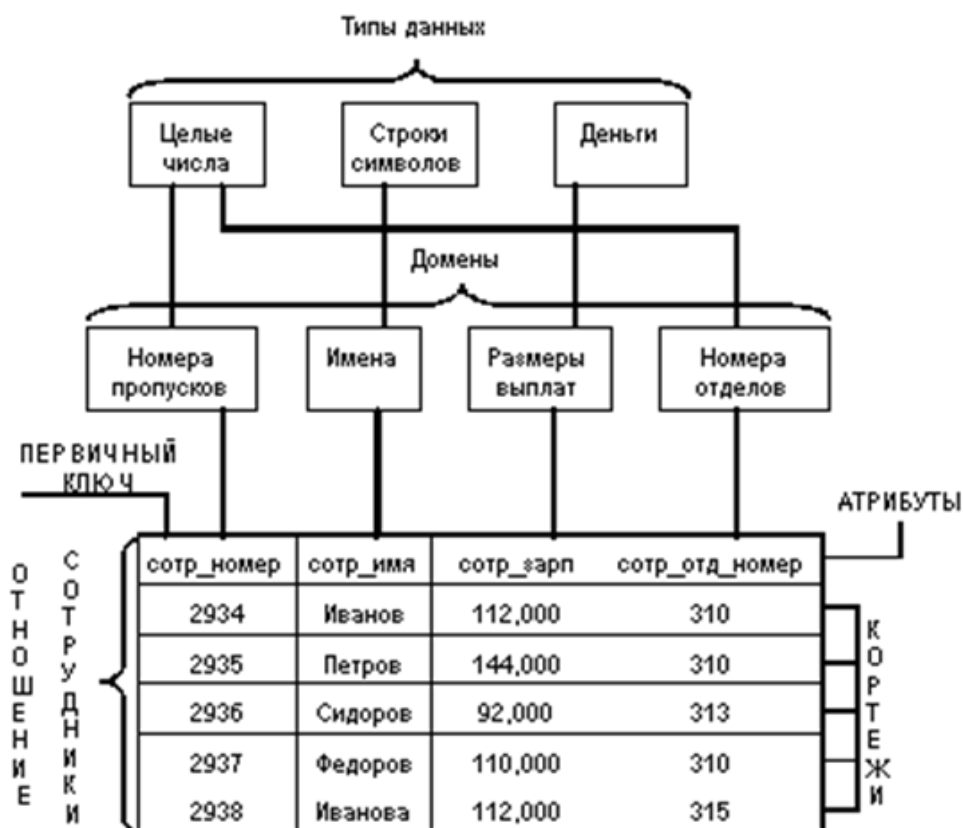


Рис. 5. Основные структурные элементы реляционной таблицы

Элементы реляционной модели

Элемент реляционной модели	Форма представления
Отношение	Таблица
Схема отношения	Строка заголовков столбцов таблицы (заголовок таблицы)
Кортеж	Строка таблицы
Сущность	Описание свойств объекта
Атрибут	Заголовок столбца таблицы
Домен	Множество допустимых значений атрибута
Значение атрибута	Значение поля в записи
Первичный ключ	Один или несколько атрибутов
Тип данных	Тип значений элементов таблицы



Первичный ключ – уникальный идентификатор, т.е. один или несколько атрибутов, значения которых в одно и то же время не бывают одинаковыми.

Например, в виде реляционной таблицы можно представить информацию о студентах, обучающихся в вузе (рис. 6).

Номер личного дела	Фамилия	Имя	Отчество	Пол	Дата рождения	Специальность
16493	Сергеев	Петр	Михайлович	м	01.01.86	080104
16593	Петрова	Анна	Владимировна	ж	15.03.85	080102
16693	Анохин	Андрей	Борисович	м	14.04.86	080104

Рис. 6. Пример реляционной таблицы

Преимущества

В отличие от других типов БД, реляционная обеспечивает целостность очень больших объемов данных. Это критически важно для операционной деятельности бизнеса, например, банковской сферы с миллионами ежедневных транзакций.

Реляционная БД предоставляет доступ для нескольких сотрудников одновременно, с возможностью настройки уровней доступа.

Недостатки

Для обеспечения целостности данных реляционная БД использует длительные и кратковременные блокировки. Это приводит к задержкам и недоступности данных на некоторое время.

Реляционная база данных требует большого объема физической памяти. Кроме того, сравнительно с другими БД, у нее невысокая производительность. И чем больше объем БД, тем медленнее обрабатываются запросы.

Организация, хранение и обработка слабо структурированных или неструктурированных данных из разных источников достаточно трудная задача.

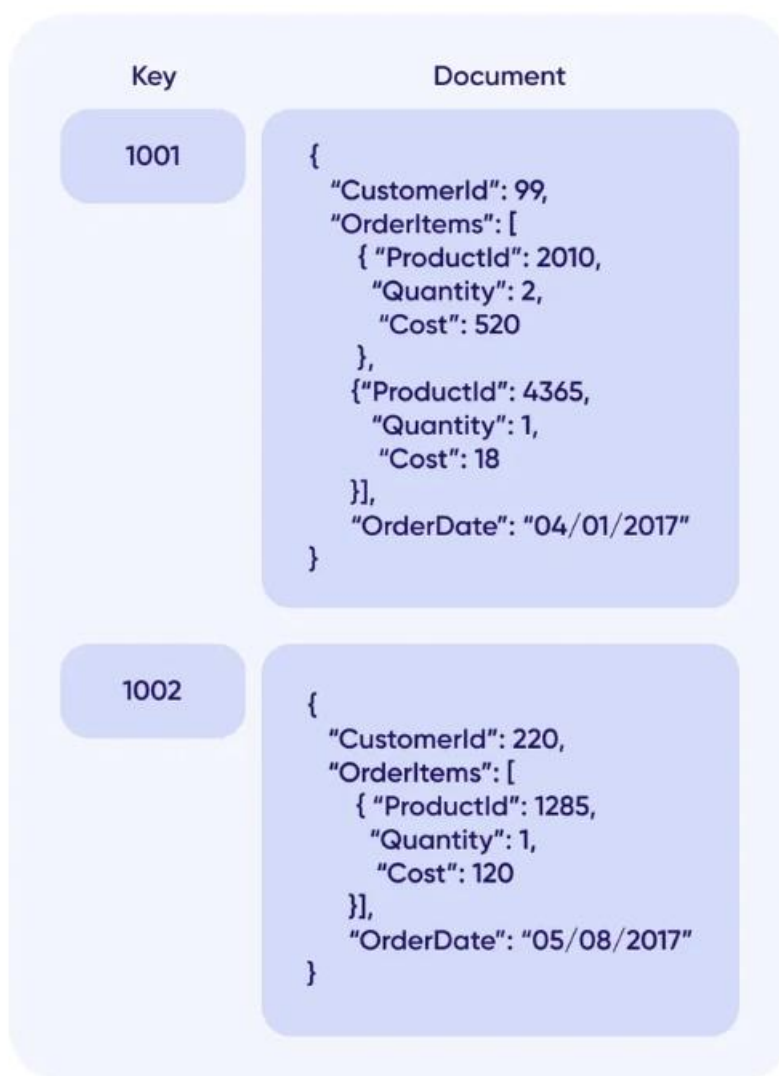
Нереляционные базы данных или базы данных NoSQL

Если сравнивать нереляционные базы с SQL, то здесь данные хранятся кардинально иным образом. Они могут не иметь какой-то четкой структуры, заранее установленной и определенной, а также четких связей. Информация хранится не в виде таблиц, а в качестве текстовых документов, файлов или публикаций в сети.

Такой формат дает возможность хранить в базах сведения любого типа, формата и объема. Именно поэтому базы активно используются в современных приложениях и программах под различное оборудование и устройства.

Документоориентированные базы данных

Такой тип БД управляет так называемым документом: набором значений и данных объекта. Важно, что один документ может содержать сведения, которые в реляционной СУБД обычно распределяются по нескольким реляционным таблицам. Вдобавок документоориентированные БД не требуют, чтобы все документы имели одинаковую структуру. Доступ к документам дается через ключ — уникальный идентификатор документа.



Колоночные базы данных

Такие БД упорядочивают данные по столбцам и строкам, подобно тому, как это делается в реляционной базе данных. Столбцы разделяются на группы, в каждую включен набор логически связанных столбцов, которые извлекаются или управляются как единое целое.

ID клиента	ФИО	ID клиента	Контактные данные
001	Имя: Иван Фамилия: Петров	001	Номер телефона: +7(999) 334-77-77 Email: Petrov@mail.ru
002	Имя: Даниил Фамилия: Романов	002	Email: Romanov@mail.ru
003	Имя: Роман Фамилия: Мухтаров	003	Номер телефона: +7(999) 777-77-77

Преимущества

NoSQL БД обладают высокой гибкостью, способны хранить обрабатывать как структурированные, так и неструктурированные данные. Кроме того, такие БД быстро масштабируются с ростом количества данных, сохраняя высокую производительность.

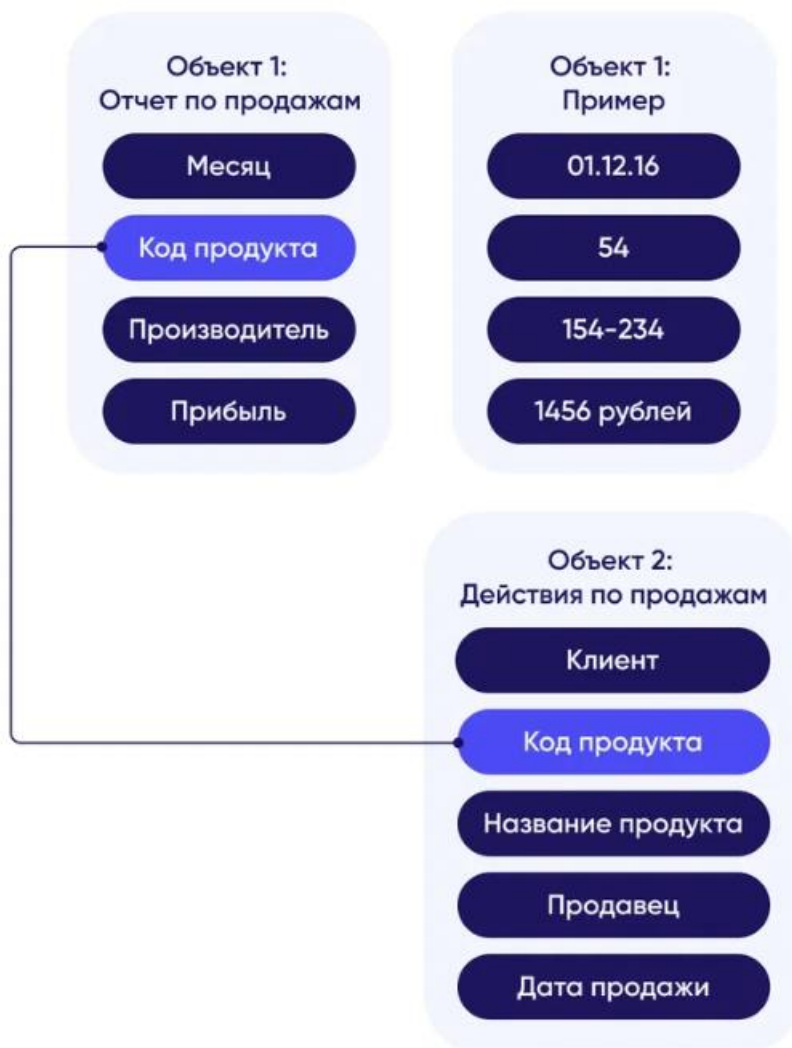
Недостатки

Отсутствие единого общепризнанного стандарта, определяющего правила и роли баз данных NoSQL. Структура и языки запросов сильно различаются в разных продуктах NoSQL.

У NoSQL БД могут быть проблемы с обработкой информации, требующей строгой согласованности во всем наборе данных.

Объектно-ориентированная база данных

Объектно-ориентированные базы данных - информация хранится в виде набора объектов или программных элементов многократного использования.



Преимущества

Объектно-ориентированная модель БД помогает точнее моделировать «реальный мир» и работать объектами, с которыми трудно справиться другим типам БД.

БД позволяют создавать новые типы данных из уже существующих, выделять общие свойства нескольких классов и сформировать из них суперкласс, который можно использовать совместно с подклассами. Это помогает уменьшить избыточность внутри системы и считается одним из основных преимуществ объектно-ориентированного подхода.

В них можно хранить самые разные типы данных, например, изображения и видео.

Недостатки

У большинства пользователей еще мало опыта работы с такими БД, вдобавок работу с ними усложняет отсутствие универсальной модели данных и общепризнанного языка запросов.

Невозможно разграничить доступ пользователей к отдельным объектам или классам.